UML 2 Models for ODP Engineering/Technology Viewpoints

Daisuke Hashimoto
Technologic Arts Inc.
hashimoto@tech-arts.co.jp

Hiroshi Miyazaki
Fujitsu, Ltd.
miyazaki.hir-02@jp.fujitsu.com

Akira Tanaka
Hitachi, Ltd.
tanakaak@itg.hitachi.co.jp

Abstract

The advance of UML® 2.0 standardization work by OMG™ provides a good opportunity for ODP community to leverage UML 2.0 to show the value of ODP. In this paper, we examine issues in applying UML 2.0 to ODP Engineering and Technology Viewpoint Languages, and show how we may be able to use UML to represent those ODP Viewpoint specifications.

1. Introduction

With the wide acceptance of Unified Modeling Language in the industry, there is a growing interest in applying UML to represent ODP viewpoint specifications. This direction is beneficial to both sides, since ODP modelers will eventually be able to get ODP modeling tool based on UML tools, and UML modelers will get a robust way of organizing their UML models. This comes from ISO/IEC and ITU-T's joint project called "Use of UML for ODP system specifications," and also comes from various "Enterprise Architecture" practices where rows in two-dimensional matrix are usually quite similar to ODP viewpoints. In this paper, we identify and examine issues in modeling Engineering and Technology Viewpoints with UML 2.0, and propose possible UML 2 model diagrams for those viewpoint specifications.

2. Engineering Viewpoint – Issues and discussion for UML modeling

2.1 Target Architectural Diagrams

First we need to identify kinds of architectural diagrams to be described in UML. Some candidates are found in RM-ODP Part 3, Clause 8 Figure 2 to 6. Figure 2 is "An example of a basic client / server channel," Figure 3 is "An example of a multi-endpoint channel," Figure 4 is "Example structure supporting a basic engineering object," Figure 5 is "Example structure of a capsule," and Figure 6 is "Example structure of a node." From those example figures, we can identify essential target diagrams, which are Channel structure diagram (covering Figure 2 and 3) and Capsule structure diagram (covering Figure 4, 5, and 6).

In addition, to cover all the Engineering concepts, we will need UML diagrams describing domains and templates.

2.2 Representation of Engineering Concepts

Engineering Object:

The issue is the choice of UML elements representing Engineering Object, which includes at least BEO, CapsuleManager, ClusterManager, Stub, Binder, ProtocolObject, and Interceptor. It could be modeled with UML Object (InstanceSpecification of Class), UML Class, UML InstanceSpecification of Component, or UML Component. In either case, UML 2's Structured Classifier will help us describe the internal structure of Channel and Capsule.

Containers:

The next are containers such as Node, Nucleus, Capsule, and Cluster. Those elements are defined as "a configurations of (basic) engineering objects …" in RM-ODP Part 3. It may be possible to use UML Node to represent ODP Node. However, if we take this approach, only available UML diagram will be Deployment Diagram, and UML 2 only allows certain types of modeling elements (e.g. Node, Artifact) to be placed within a Node. Note that it was possible to take this approach with UML 1.4, since it was legal to place Component within a Node. The UML diagram we need has to have a capability to show the internal and logical structure of ODP Node, similar discussion regarding Class or Component in Engineering Object may also apply.

Nucleus:

Nucleus may either be treated as Engineering Object or container. It could be Engineering object, since it provides Nucleus Services to all types of Engineering Objects. It could also be considered as a container, in which Capsule and Containers may reside within.

Channel:

Channel may either be treated as a container or a structured Object or Class etc. The issue associated with Channel is that

both Channel and Capsule shares the same Engineering Objects (Stub, Binder, and ProtocolObject) and we do not have overlapping diagrams even in UML 2.0.

Template:

<X> Template is defined in RM-ODP Part 2 as "The specification of the common features of a collection of <X>s in sufficient detail that an <X> can be instantiated using it. <X> can be anything that has a type." The templates in Engineering Viewpoint Language are Cluster template, Checkpoint, Cluster checkpoint. However, the definitions of those Engineering templates seem more like "snapshot" than the Part 2 definition implies. The choice of UML model element for template has impact on the choice of UML model element for Engineering Object.

2.3 Interactions between Engineering Objects

In Computational Viewpoint, RM-ODP Part 3 provides various concepts to deal with interactions with interface and signature. In Engineering Viewpoint, however, we do not have specific concepts for this purpose, and that makes it difficult to model interactions between Engineering Objects. One possibility is to consider "recursive application of viewpoints." If we can apply Computational Viewpoint Language to interactions between BEOs, we will get a capability to specify this. In that case, the modeling element will represent Engineering Object with Computational aspects. The issue becomes how to do this in UML. For instance, do we want to allow UML Class or Component stereotyped as "«NV_BEO» «CV_Object»?"

In addition, if we are to allow "recursive application of viewpoints" as described here, we would need to consider the alignment of base classes for corresponding viewpoint profile.

2.4 Engineering Functions

In Engineering Viewpoint, there are functionalities defined as a part of its language. Those are Checkpointing, Deactivation, Cloning, Recovery, Reactivation, and Migration. Those are different from other concepts, since they are representing functions possibly including behaviors. One possibility is to apply "recursive application of viewpoints" again, and make use of Enterprise Viewpoint's Objective and Process etc. concepts to model those functions as «EV_Objective» with behaviors as «EV_Process» expressed with Activity Diagram.

2.5 ODP functions

In RM-ODP Part 3, many ODP functions are described. When defining ODP viewpoint specifications, those common functions may need to be explicitly included. An example of ODP function is ODP Trader, where the ODP Trader standard defines Enterprise, Information, and Computational Viewpoint specifications of itself. The issues are how we can identify, reference, and include those functions in Engineering Viewpoint specifications.

2.6 Domains

Domain concept in RM-ODP is defined as follows. <X> Domain is "A set of objects, each of which is related by a characterizing relationship <X> to a controlling object. Every domain has a controlling object associated with it." It is a set of objects, rather than a configuration of objects. The issue is which UML element is suitable for representing this "a set of objects" concept.

Also, since the same Engineering Object may be a member of several different kinds of domains at the same time (e.g. NamingDomain_1, SecurityDomain_2, and PolicyDomain_3 etc.), UML element representing domain should allow sharing of objects. If we choose Class, parts may be shared. If we choose Component, it may become issues. If we choose Package, we will need to use «import»/«access» etc. to have access to those elements contained in other Packages.

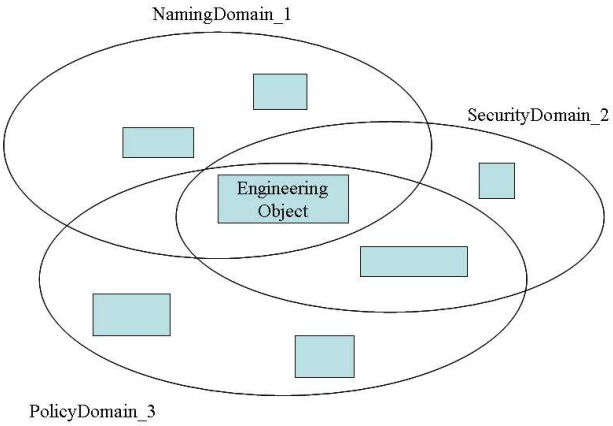


Figure 1 Engineering Objects and Domains

2.7 Selective Transparencies

Computational Viewpoint specifications are defined in distribution transparent manner, and the degree of distribution transparency may be specified in a "transparency schema" associated with "a specification that uses specific ODP functions and engineering structures to provide the required form of masking." This may be considered as a mapping specification from Computational Viewpoint to Engineering Viewpoint. And, this implies that given one Computational Viewpoint specification, there will be a variety of Engineering Viewpoint

specifications, each corresponding to different transparency schema. Although this issue may be mainly related with Computational Viewpoint, it has an impact on Engineering Viewpoint specification and Engineering Viewpoint's correspondence to Computational Viewpoint specification.

2.8 Architectural Styles (or Engineering Templates or Patterns)

Since the day RM-ODP Part 3 became International Standard, five or more years have passed, and now there are various types of commercial and open source middleware which in fact implements most of the Engineering Viewpoint concerns, e.g. CORBA ORBs, J2EE application servers. .NET, Web Services, SOA, and Message Oriented Middleware, etc. Even more, there are new types of middleware emerging for Grid and Utility Computing, Wireless Networking, and Collaboration Environment etc. Also, various best practices and architectural styles (such as MVC) were developed and in use. Now may be a good time to define architectural styles as Engineering Templates and publish them for use in mapping Computational Viewpoint specifications to Engineering Viewpoint specifications. The issue is how to define those architectural styles with UML.

3. Engineering Viewpoint – Possible UML 2 Models

3.1 Profile definition

The first step is to decide which UML element should be used to represent Engineering Viewpoint Language, especially Engineering Object. We also need to consider about related templates. The choice we made in this paper is use of UML Component for Engineering Objects. A Component is closer to real world software component than Class, and UML 2 provides Component with structuring capability also. The following is a Class diagram showing partial UML 2 Profile definition.

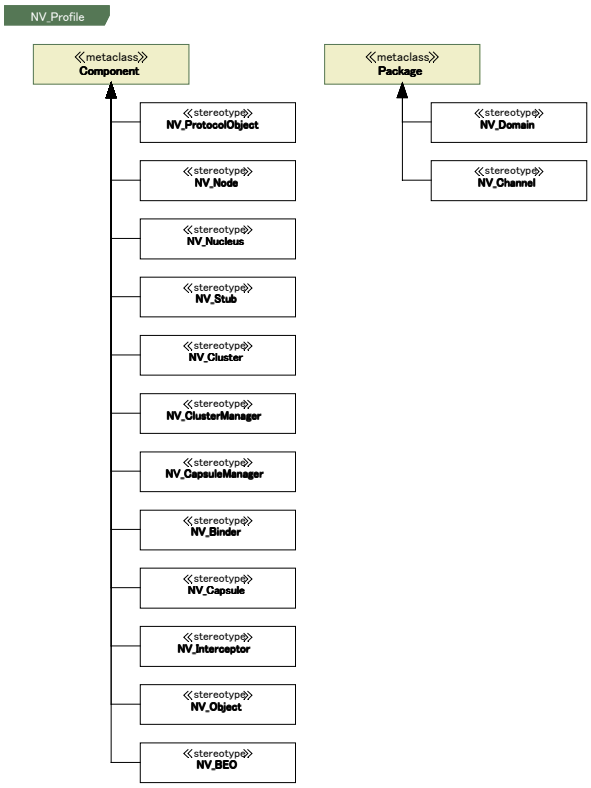


Figure 2 UML 2 Profile for Engineering Viewpoint Language

3.2 Example UML Diagrams

As discussed before, we need to be able to model structure of Node etc. Structural UML Diagram showing Node, Capsule, Clusters, and BEOs is shown next in Figure 3. The diagram is based on Composition Diagram of UML 2 and Component is used to represent most of the Engineering concepts including containers. Some interesting observations are whether Capsule and Cluster should have interfaces or not, since Capsule is a unit of resource assignment, Cluster is a unit of activations and deactivations, and also they have associated Managers. If they should have interfaces, then what kind of interface should they be?

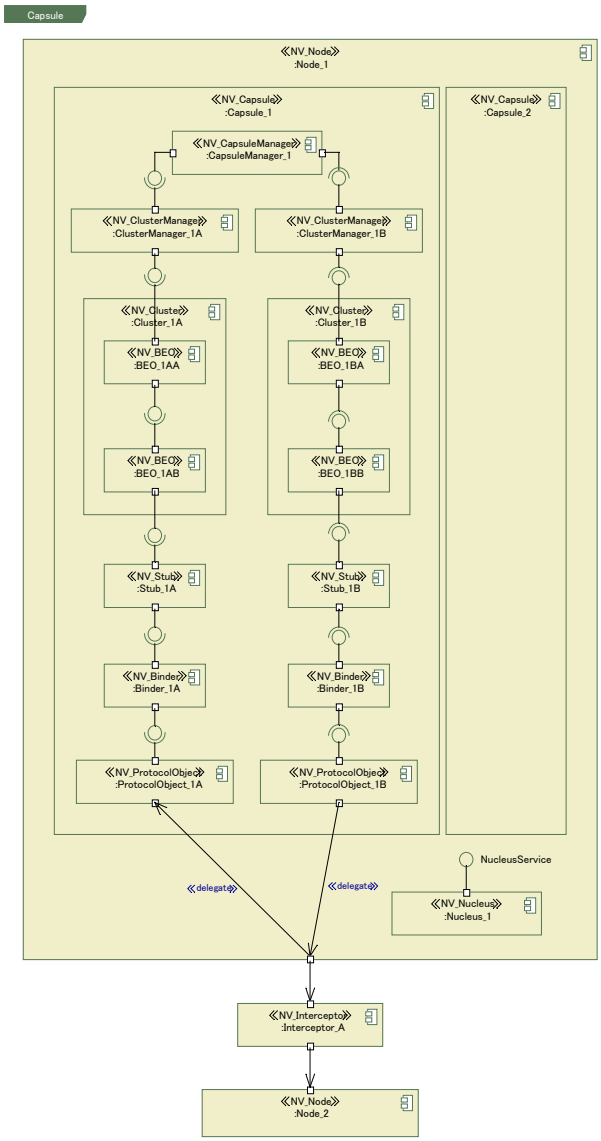


Figure 3 Example UML 2 model for Node, Capsule, Cluster, and BEO

Channel may be represented with the following UML 2 diagram. The same Engineering Objects appear in Figure 3 and Figure 4 (i.e., Stub, Binder, Protocol Object and Interceptor). Figure 3 shows the structure, interfaces with ports, and services in the hierarchy. Figure 4 shows membership of the Channel Package. Those two complements each other to provide different views to the same set of Engineering Objects making up a part of Node and Channel.

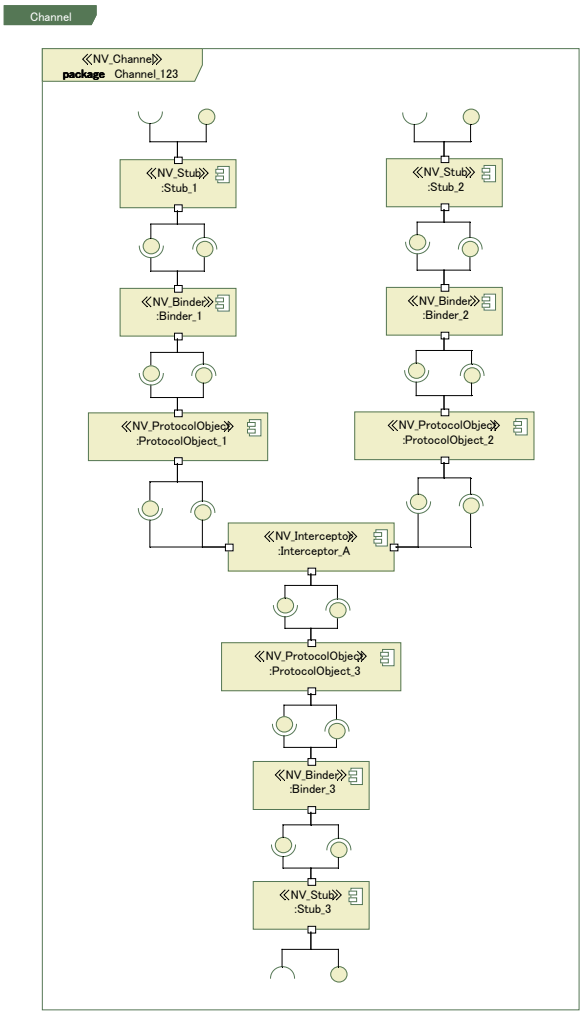
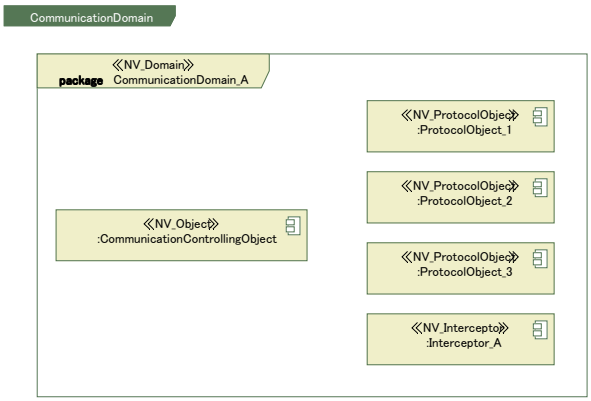


Figure 4 Example UML 2 model for Channel

Domain may be represented with the following UML 2 diagrams (Packages). One missing element is a Policy that Controlling Object is enforcing. Since Policy concept, at least the base class for it, should be aligned with other viewpoint (i.e. Enterprise Viewpoint), and that is not concluded yet, we did not include it in this diagram.



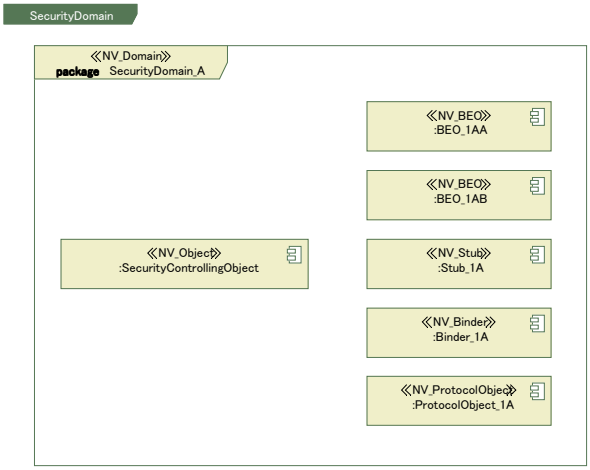


Figure 5 Example UML 2 model for Domain

Template in Engineering Viewpoint may be represented with structural diagram with InstanceSpecification (Component) diagram as snapshot at certain location in time.

3.3 Correspondence

From Engineering Viewpoint to Computational Viewpoint

Our assumption is that each BEO has one to one relationship to corresponding Computational Object. This could be expressed as dependency from BEO sub-Package in Engineering Viewpoint Package to Computational Objects in Computational Viewpoint Package in UML. Note that the distribution support mechanism part of Engineering Viewpoint Language model elements may have one to one relationship with specified transparencies in transparency schema.

From Engineering Viewpoint to Technology Viewpoint

This correspondence is closely related to OMG's MDA initiative where Platform Independent Model is transformed to Platform Specific Model. In this case, Platform means real platform software and hardware. It is our expectation that when MDA standards arrive, they will be considered as technology related to Viewpoint Correspondence.

Each Engineering Objects may be implemented with multiple Technology Objects, and multiple Engineering Objects may be implemented with one Technology Object. This could be expressed also as dependency between two Viewpoint Packages.

4. Technology Viewpoint – Issues and discussion for UML modeling

4.1 Hardware, Software and Network

In UML Deployment Diagram, main modeling elements are Node and Artifact. Typical representation of hardware elements such as CPU and memory is by the use of tagged values to Node, and we follow this approach. For operating systems and middleware, UML 2 introduced new metaclass ExecutionEnvironment to Node. We can add, if necessary, ODP semantics to this model element. Software implementing BEOs can be modeled as Artifact. CORBA Middleware, for instance, and Operating System, POSIX-compliant OS for instance, may be modeled based on ExecutionEnvironment. Network like the Internet can be modeled as a Node, although hardware/software aspects of network can be treated differently.

4.2 Representation of Technology Concepts

Technology Object

Technology Objects, covering software, hardware and network, may be represented with UML Artifact or Node (including Node within Node).

Implementable standard may be considered as a part of specification Technology Objects implements.

Implementation is defined as "a process of instantiation whose validity can be subject to test." A process may be represented with Activity Diagram.

IXIT (Implementation eXtra Information for Testing) may be represented with UML Comments as annotations to Technology Viewpoint specifications.

5. Technology Viewpoint – Possible UML 2 Models

5.1 Profile definition

The following is a diagram showing a partial UML 2 Profile definition. In order to represent hardware, software, and network as Technology Object, two base classes are used for Technology Object.

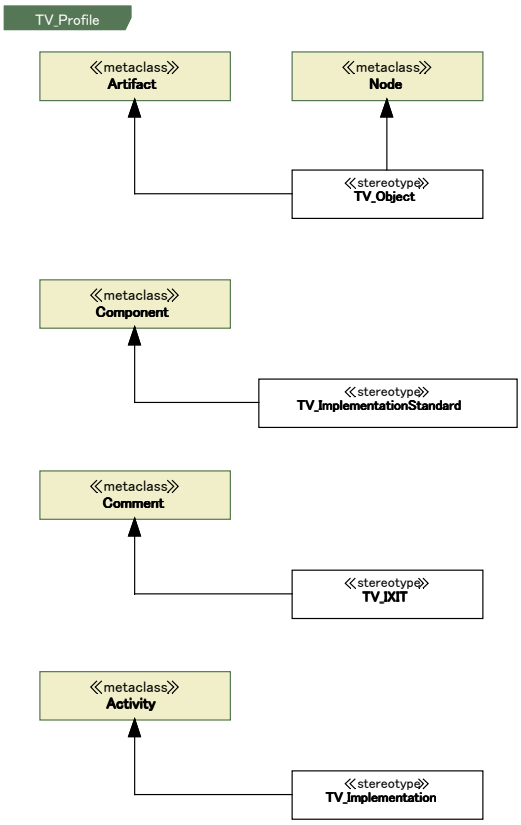


Figure 6 UML 2 Profile for Technology Viewpoint Language

5.2 Example UML Diagram

The following is a diagram showing physical structure of a node with hardware, software, and network elements.

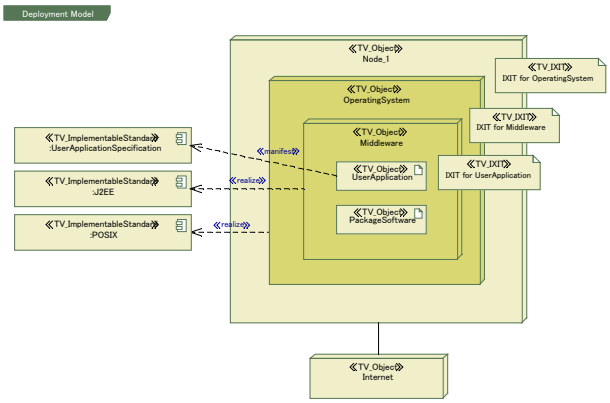


Figure7 Example UML 2 model for Technology Viewpoint

5.3 Correspondence

<u>From Technology Viewpoint to Engineering Viewpoint</u>

Each Technology Object has one to one relationship to corresponding Engineering Object. This could be expressed as dependency from Technology Objects in Technology Viewpoint Package to Engineering Objects in Engineering Viewpoint Package.

6. Conclusions

There may be multiple ways to represent ODP viewpoint specifications with UML 2.0. We believe we have choices e.g. on abstract vs. instance, and class vs. component. And, one important aspect is being practical and usable in real world. We hope that UML Profile for ODP to be standardized soon and the profile data be developed, published, and become openly available to interested parties, especially ODP modelers.

Acknowledgements

The work described here was mostly done within INTAP Open Distributed Processing Committee, and includes discussion items from Japanese Committee for SC7/WG19.

References

[1] RM-ODP, ITU-T Recommendation X.901 to X.904 and ISO/IEC 10746-1 to 4
http://isotc.iso.ch/livelink/livelink/fetch/2000/2489/Ittf_Home/ITTF.htm??Redirect=1

[2] UML 2.0 Superstructure, OMG,
http://www.omg.org/cgi-bin/doc?ptc/2004-10-02

[3] Model Driven Architecture, Document number: ormsc/01-07-01

[4] MDA Guide, Document number: omg/03-06-01